실습: Solidity (2)

블록체인, 2026-2학기



김효승 (Hyoseung_kim@hallym.ac.kr)
정보과학대학 소프트웨어학부

- **학습목표**
  - 배열의 종류를 이해하고 간단한 기능을 구현한다
  - 구조체의 개념과 사용법을 이해한다
  - 매핑의 특성을 이해하고 구현한다
  - 전역 변수와 함수의 사용법을 이해한다
  - 에러 처리의 쓰임을 구분한다
  - 함수 제한자를 정의하고 함수에 사용한다
  - 이벤트를 정의하여 상태 변화를 기록하고 확인한다
  - 상속 규칙을 이해한다

- **배열**
  - 컴파일 시점에 고정된 크기, 동적 크기 가질 수 있음
    - 고정된 크기는 T[k], 동적크기는 T[] 와 같이 선언됨

```
uint[5] fArray = [1, 2, 3, 4, 5];
uint[] dArray;
```

- uint[][3]: 동적 배열로 구성된 **크기가 3인 uint 타입 배열**
  Ex) [[1,2,3,4,5], [6,7], [10,100,1000]]

- bool[2][] : **크기가 2인 배열**로 구성된 bool 타입 동적 배열
  EX) [[true,false], [false,true], [true, true], [false, false]]

- 배열
  - 예시)

```
bool[2][] flagArray;
function setFlagArray(bool[2][] memory _flagArray) public {    infinite gas
    flagArray = _flagArray;
}

// calldata 형식: [[true,false],[false,true], [true, true]] (길이가 2인 배열)의 배열
function setFlagPair(uint index, bool flagA, bool flagB) public {    infinite gas
    flagArray[index][0] = flagA;
    flagArray[index][1] = flagB;
}
```

```
int[][2] intArray;
function setIntArray(int[][2] memory _intArray) public {    infinite gas
    intArray = _intArray;
}

// calldata 형식: [[1,2,3,4,5],[6,7]] 길이가 2인 (배열의 배열)
function setIntValues(uint indexOfArray, uint indexOfUnit, int x) public {    infinite gas
    intArray[indexOfArray][indexOfUnit] = x;
}

function intFlagArray() public view returns(int[][2] memory){    infinite gas
    return intArray;
}
```

- **문자열(String)과 바이트(Bytes) 배열**
  - bytes 동적 배열은 참조 타입으로 동작
    - 동적 배열은 push() 함수를 이용하여 마지막 요소를 추가할 수 있음
  - string은 bytes와 유사하지만 index 접근은 안됨
    - EVM에서는 두 타입 모두 바이트 배열로 구현

```
contract BytesAndString{
    string internal constant TITLE = "HELLO";
    bytes internal bytesTitle = bytes(TITLE);

    function getTitleString() public pure returns (string memory){
        return TITLE;
    }

    function getTitleBytes() public view returns (bytes memory, uint){
        return (bytesTitle, bytesTitle.length);
    }

    function appendTitle() public {
        for(uint i = 0; i < bytesTitle.length; i++){
            bytesTitle[i] = 'a';
        }
        bytesTitle.push('!');
        bytesTitle = bytes.concat(bytesTitle, bytes("@@"));
    }
}
```

constant 값은 pure로 읽기 가능

infinite gas

- **구조체**
  - struct 키워드로 문자열, 정수 등의 변수를 그룹화 할 수 있는 사용자 정의 유형
    - 예시1)

```
struct Voter{
    uint weight;      // 투표권의 가중치
    bool voted;       // 이미 투표했는지 여부
    address delegate; // 투표권을 위임받은 주소
    uint vote;        // 실제 선택한 투표 항목(번호)
}
```

    - 예시2)

```
contract Structs{
    struct User {
        string name;
        string email;
    }

    User[] public users;

    function addUser(string calldata _name, string calldata _email) public {
        users.push(User(_name, _email));
    }
}
```

- **매핑 (Mapping)**
  - mapping(keytype => valuetype) 으로 정의하며 크기 제한이 없음
    - Keytype: 모든 기본값 (int, bytes, string, …) 가능 but 다른 맵핑, 구조체, 동적 배열 불가
    - Valuetype: 어떤 타입이든 가능
  - 스토리지에만 선언 가능
  - 지역 변수 사용 시 참조 형식으로만 가능
    - 매핑 전체는 삭제(delete) 불가능
    - 키 단위로는 삭제 가능

```
contract Mappings{
    struct User {
        string name;
        string email;
    }

    mapping(address => User) internal users;

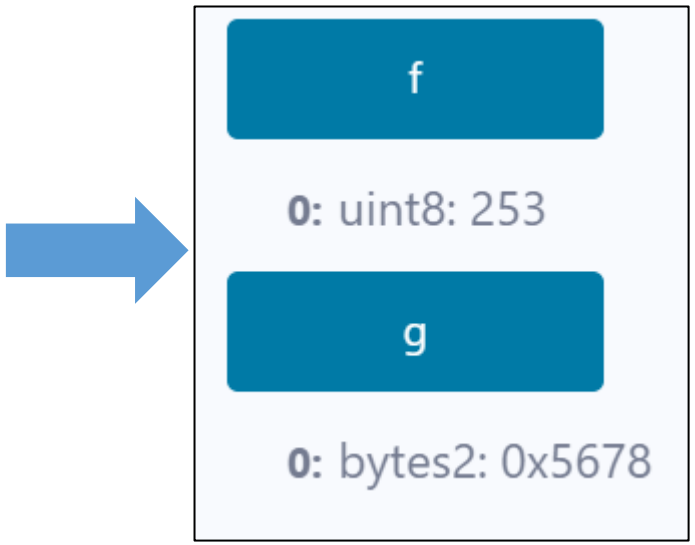
    function addUser(string calldata _name, string calldata _email) public{
        users[msg.sender].name = _name;
        users[msg.sender].email = _email;
        // users[msg.sender] = User({name: _name, email: _email});
    }

    function getUser() public view returns(string memory, string memory){
        return (users[msg.sender].name, users[msg.sender].email);
    }
}
```

- **타입 캐스팅**
  - 크기가 작은 타입을 큰 타입으로 변경하는 의미
    - 반대로 캐스팅 할 시 자료 손실 발생할 수 있음
    - 일반적으로 컴파일러가 자동으로 캐스팅하지만 개발자가 직접 명시할 수 있음

```
contract Conversions {
    function f() public pure returns(uint8){
        int8 y = -3;
        uint8 x = uint8(y);
        return x;
    }

    function g() public pure returns (bytes2){
        uint32 a = 0x12345678;
        uint16 b = uint16(a);
        return bytes2(b);
    }
}
```



- **통화 단위**
  - 1 wei (기본)
  - 1 gwei = 10^9 wei
  - 1 ether = 10^{18} wei
- **시간 단위**
  - 1 seconds = 1초
  - 1 minutes = 60 seconds
  - 1 hours = 60 minutes
  - 1 days = 24 hours
  - 1 weeks = 7 days
  - 1 year = 365 days

```
contract Units{
    function f() public pure returns(uint a, uint b, uint c){
        a = 1 wei;
        b = 1 gwei;
        c = 1 ether;
    }

    function g(uint start, uint daysAfter) public view {
        if (block.timestamp >= start + daysAfter * 1 days) {
            //...
        }
    }
}
```

- **블록, 메시지, 트랜잭션 관련 변수**

변수 이름	반환	설명
blockhash(uint blockNumber)	bytes32	최근 **256개** 블록 범위에서 해당 블록의 해시를 반환 (범위 밖이면 0x0)
block.basefee	uint	현재 블록의 기본 수수료
block.chainid	uint	현재 체인 ID(멀티체인 대응 시 활용)
block.coinbase	address payable	현재 블록의 검증자(수수료 수령자) 주소
block.gaslimit	uint	현재 블록의 가스 한도
block.number	uint	현재 블록 번호
block.timestamp	uint	현재 블록 유닉스 타임스탬프
gasleft()	uint256	현재 실행 컨텍스트에 남은 가스
msg.sender	address	현재 호출자 주소
msg.value	uint	호출과 함께 전송된 wei 금액
tx.gasprice	uint	트랜잭션의 유효 가스 가격(basefee + tip)
tx.origin	address	트랜잭션의 최상위 발신자 주소

- ## 수학 및 암호화 함수

시그니처(입력)	반환	설명
addmod(uint x, uint y, uint k)	uint	((x + y) mod k)를 오버플로 없이 계산
mulmod(uint x, uint y, uint k)	uint	((x × y) mod k)를 오버플로 없이 계산
keccak256(bytes memory)	bytes32	솔리디티 표준 해시
sha256(bytes memory)	bytes32	비트코인·표준 시스템과의 호환 목적
sha3(bytes memory)	bytes32	NIST 표준 해시
ripemd160(bytes memory)	bytes20	비트코인 주소 축약 및 특정 프로토콜 호환 목적
ecrecover(bytes32 hash, uint8 v, bytes32 r, bytes32 s)	address	ECDSA 서명에서 서명자 주소 복구 실패 시 address(0) 반환
abi.encode(...)	bytes	컨트랙트와 외부 간 데이터 전송을 위한 ABI 표준 인코딩 (패딩 포함)

- **가시성**
  - Public: 모든 컨트랙트 및 계정에서 호출 가능
  - Private: 컨트랙트 안에서만 사용 가능
  - Internal: 컨트랙트 안 또는 상속받은 파생 컨트랙트에서만 호출 가능
  - External: 다른 외부 컨트랙트·계정에서 호출 가능 (내부에서 사용 불가능)

- **속성**
  - view: 블록체인의 상태변수가 변경되지 않음 (블록체인 데이터를 읽기만 할 경우)
  - pure: 블록체인의 상태변수가 변경되지 않고, 읽지도 않음

- **제어 구조**
  - if, else, while, do ,for, break, continue, return 사용 가능
  - switch와 goto는 없음

- **내부·외부 함수 호출**
  - 내부재귀 함수는 문법적으로 호출 가능하나 가스 한도 내에서 실행
  - (외부) 컨트랙트/인터페이스 타입으로 인자 받아 실행

```
contract A {
    bool public flag;

    function setFlag(bool _flag) external {    24650 gas
        flag = _flag;
    }
}

contract B {
    function callExternal(address aAddress, bool value) public {
        A(aAddress).setFlag(value);
    }
}
```

```
contract FunctionCall{
    function pub() public pure returns (uint) {
        return 100;
    }

    function ext() external pure returns (uint) {
        return 200;
    }

    function test() public view returns (uint, uint, uint) {
        uint a = pub();       // 가능: 내부 호출
        // uint b = ext();    // 불가능: 컴파일 오류    310 gas
        uint b = this.ext();  // 가능: 외부 호출
        uint c = this.pub();  // pub()의 외부 호출    332 gas

        return (a, b, c);
    }
}
```

- **에러 처리**
  - require(bool) : 조건 안 맞을 시 되돌려짐, 입력, 외부구성요소의 오류에 사용
  - require(bool, string) : 위와 동일, 오류메세지 제공
  - revert() : 실행을 중단하고 상태 변경사항을 되돌림
  - revert(string) : 위와 동일, 오류메세지 제공
  - assert(bool): 내부 상태 에러가 없는지 확인, 에러 발생 시 모든 가스 소비

```
contract ErrorHanding {
    function sendHalf(address addr) public payable returns(uint balance){
        // 2로 나눌 수 있는 금액만 전송
        require(msg.value % 2 == 0, unicode"유효 금액이 아닙니다.");
        uint balanceBefore = address(0).balance;
        payable(addr).transfer(msg.value / 2);

        // 내부 오류가 없는지 확인
        assert(address(0).balance == balanceBefore - msg.value / 2);

        return address(0).balance;
    }
}
```

- **제한자 (Modifier)**
  - 함수 실행 전후에 동작을 자동으로 추가
    - 코드 중복을 줄이고 유지 보수성을 높임
  - 주로 접근제어, 사전 조건 검사, 재진입 방지 등에 사용

```
contract Modifier {

    address public owner;

    constructor(){    201282 gas 176800 gas
        owner = msg.sender;
    }

    modifier onlyOwner() {
        // 함수 실행 전 조건
        if(msg.sender != owner) revert(unicode"컨트랙트 소유자만 호출 가능");

        // 함수 실행 위치
        _;

        // 함수 실행 후 조건
        assert(owner != address(0));
    }

    function changeOwner(address _owner) public onlyOwner {
        owner = _owner;    29075 gas
    }
}
```

함수 제한자

- **이벤트 (Event)**
  - 트랜잭션 로그 영수증(receipt)에 저장하고 출력
  - 블록체인에 데이터를 가장 저렴하게 기록할 수 있음
    - 주로 로그를 남기기 위해 사용

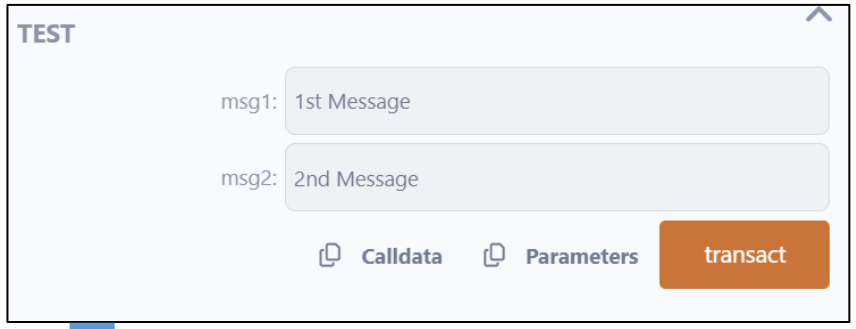
```
contract Events{
    event Log(address sender, string message);
    event AnotherLog();

    function test(string memory msg1, string memory msg2) public{
        emit Log(msg.sender, msg1);
        emit Log(msg.sender, msg2);
        emit AnotherLog();
    }
}
```

- 이벤트
  - 예시

```
contract Events{
    event Log(address sender, string message);
    event AnotherLog();

    function test(string memory msg1, string memory msg2) public{
        emit Log(msg.sender, msg1);
        emit Log(msg.sender, msg2);
        emit AnotherLog();
    }
}
```



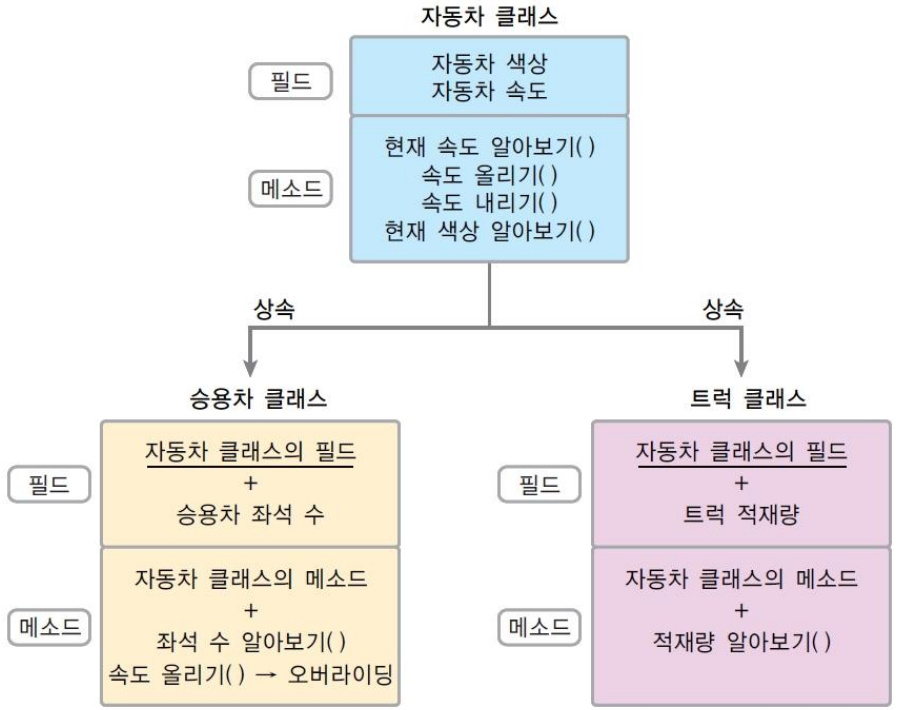
Output >_ 0 Listen on all transactions Filter with transaction hash

decoded output {}

logs

```
[
        {
                "from": "0xc2955A581b9481B8e56374dBf78adDA30D7dEc11",
                "topic":
"0x0738f4da267a110d810e6e89fc59e46be6de0c37b1d5cd559b267dc3688e74e0",
                "event": "Log",
                "args": {
                        "0": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4",
                        "1": "1st Message"
                }
        },
        {
                "from": "0xc2955A581b9481B8e56374dBf78adDA30D7dEc11",
                "topic":
"0x0738f4da267a110d810e6e89fc59e46be6de0c37b1d5cd559b267dc3688e74e0",
                "event": "Log",
                "args": {
                        "0": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4",
                        "1": "2nd Message"
                }
        },
        {
```

- **상속**
  - is 키워드를 이용하여 상속할 수 있으며 여러 부모를 다중 상속도 가능
  - 오버라이딩 허용
    - 부모 컨트랙트에서 virtual 키워드 이용
    - 제한을 강화하는 방향으로 변경 가능 Ex) (부모) view → (자식) pure



- 상속
  - 예시

```
contract A {
    function f() public pure virtual returns(string memory){    infir
        return "A";
    }
}

contract B is A {
    function f() public pure virtual override returns(string memory){
        return "B";
    }
}

contract C is A {
    function f() public pure virtual override returns(string memory){
        return "C";
    }
}

contract D is B,C { // 오른쪽 우선 순위
    function f() public pure override(B, C) returns(string memory){
        return super.f();
    }
}
```

- **참고 강의교안**
  - 처음 배우는 블록체인| 가사키 나가토, 시노하라 와타루 | 한빛미디어
  - Mastering Ethereum | Andreas M. Antonopoulos 외 4인 | O'Reilly Media

Thank You

Q&A